RESEARCH LITERATURE RECOMMENDATION SYSTEM AND ACADEMIC DOMAIN PREDICTOR

CS683 (*Natural Language Processing*)

Navya Dhawde (2101129)
Akshat Lohia (2101024)
Prateek (2101151)

Introduction

This project comprises two main components:

- **Recommendation System:**
  - Provides a list of related academic papers based on a given paper title.
  - Leverages embeddings generated by a pretrained **sentence transformer** model to capture semantic similarities between research papers.

- **Domain Predictor:**
  - A multi-label classification model that predicts relevant research categories.
  - Utilizes paper titles and abstracts to identify and categorize academic domains effectively.

Dataset

Dataset Source:

- **Kaggle's "arxiv-paper-abstracts" dataset**
  This dataset comprises abstracts and metadata of research papers, providing a rich resource for academic domain classification.

Key Features:

- **Title**: The title of the research paper, which often conveys the main topic.
- **Abstract**: A brief summary of the research.
- **Categories (Domains)**: The classification of papers into various academic domains.

Data Preprocessing

Steps Taken:

- **Loading the Data:** The dataset is loaded into a Pandas DataFrame from a CSV file.
- **Filtering Unique Titles:** Duplicate titles are removed to ensure each title is unique.
- **Exploding Terms:** The terms associated with each abstract are converted from lists into individual entries for further processing.
- **Grouping Terms:** Papers with only one associated term are filtered out, retaining those with multiple terms.

Train-Test Split

The data is split into training, validation, and test sets, with stratification to maintain the distribution of terms.

DOMAIN PREDICTOR

Data Preparation

Creating a String Lookup Layer: This layer converts the terms into a binary multi-hot encoded format, allowing the model to handle multi-label classification.

```
terms = tf.ragged.constant(train_df['terms'].values)
lookup = tf.keras.layers.StringLookup(output_mode='multi_hot')
lookup.adapt(terms)

vocab = lookup.get_vocabulary()

print("Vocabulary:\n")
print(vocab)
```

```
In [15]: sample_label = train_df["terms"].iloc[1]
         print(sample_label)
         label_binarized = lookup([sample_label])
         print(f"Label-binarized representation: {label_binarized}")

['stat.ML']
Label-binarized representation: [[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]]
```

Creating TensorFlow Datasets

Here we prepare TensorFlow datasets for text classification by batching and optionally shuffling data from DataFrames. The make_dataset function converts label data to a binarized format and pairs it with corresponding text. It shuffles training data but not validation/test sets, enabling efficient batching and preprocessing for model training, validation, and testing.

```
def make_dataset(dataframe, is_train=True):
    labels = tf.ragged.constant(dataframe["terms"].values)
    label_binarized = lookup(labels).numpy()
    dataset = tf.data.Dataset.from_tensor_slices((dataframe["abstracts"].values, label_binarized))
    dataset = dataset.shuffle(batch_size * 10) if is_train else dataset
    return dataset.batch(batch_size)
```

In [17]:

```
train_dataset = make_dataset(train_df, is_train=True)
validation_dataset = make_dataset(val_df, is_train=False)
test_dataset = make_dataset(test_df, is_train=False)
```

Vocabulary Creation

In [19]:

```
vocabulary = set()
train_df['abstracts'].str.lower().str.split().apply(vocabulary.update)
vocab_size = len(vocabulary)
```

Text Vectorization Layer Setup

Initializes `Text Vectorization` with TF-IDF encoding and applies bi-gram tokenization.
Adapts the vectorizer to the text in `train_dataset`.

```
text_vectorizer = layers.TextVectorization(max_tokens=vocab_size,ngrams=2,output_mode="tf_idf")

text_vectorizer.adapt(train_dataset.map(lambda text, label: text))
```

Apply Text Vectorization to Datasets

Maps text_vectorizer to the train, validation, and test datasets to transform abstracts into vectorized form.

```
train_dataset = train_dataset.map(lambda text, label: (text_vectorizer(text), label), num_parallel_calls=auto).prefetch(auto)
validation_dataset = validation_dataset.map(lambda text, label: (text_vectorizer(text), label), num_parallel_calls=auto).prefetch(auto)
test_dataset = test_dataset.map(lambda text, label: (text_vectorizer(text), label), num_parallel_calls=auto).prefetch(auto)
```

Model Architecture

The model is a simple feedforward neural network constructed using Keras. The architecture consists of:

1. **Input Layer**: A dense layer that takes the processed input vectorized text.

2. **Hidden Layers**: Two dense layers. Dropout layers are included to prevent overfitting by randomly dropping units during training.

3. **Output Layer:** A final dense layer with a sigmoid activation function, producing a binary output for each term in the vocabulary, indicating the presence or absence of each label.

```
model1 = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(lookup.vocabulary_size(), activation='sigmoid')
])
```

Model Compilation

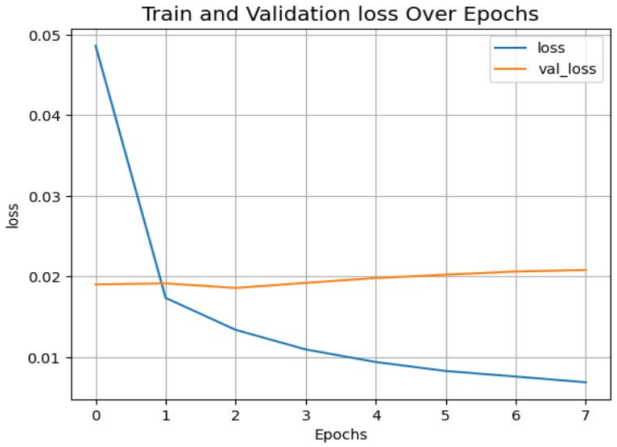
The model is compiled using the binary cross-entropy loss function, which is appropriate for multi-label classification problems. The training process is monitored using the EarlyStopping callback to avoid overfitting.

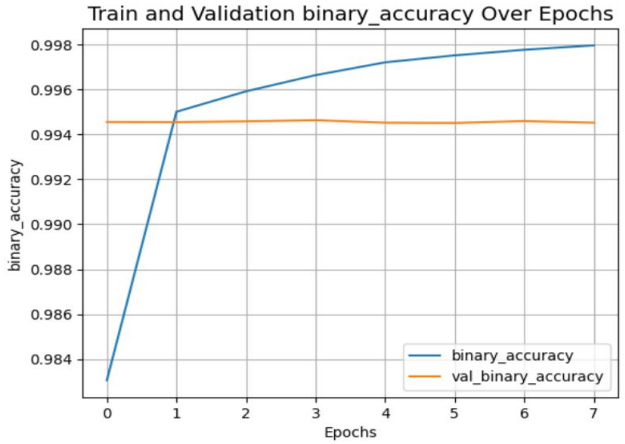
```
model1.compile(loss="binary_crossentropy", optimizer='adam', metrics=['binary_accuracy'])
es = EarlyStopping(patience=5, restore_best_weights=True)

history = model1.fit(train_dataset, validation_data=validation_dataset, epochs=20,
callbacks=[es])
```

Model Training and Evaluation Results

After training, the model's performance is visualized through loss and accuracy plots, helping assess how well the model trained over epochs.





Model Evaluation Results

16/16 ━━━━━━━━━━━━━━━━━━━━ **11s** 172ms/step - binary_accuracy: 0.9945 - loss: 0.0180

16/16 ━━━━━━━━━━━━━━━━━━━━ **3s** 146ms/step - binary_accuracy: 0.9945 - loss: 0.0188

Categorical accuracy on the test set: 99.44%.

Categorical accuracy on the validation set: 99.46%.

Demo and Predictions

To make predictions on new abstracts, the model utilizes the loaded vectorizer to preprocess the input and predict the associated categories. The prediction function outputs the predicted terms based on the model's output.

```
def predict_category(abstract, model, vectorizer, label_lookup):
    preprocessed_abstract = vectorizer([abstract])

    predictions = model.predict(preprocessed_abstract)
    predicted_labels = label_lookup(np.round(predictions).astype(int)[0])

    return predicted_labels
```

In [33]:

```
new_abstract = 'Deep networks and decision forests (such as random forests and gradient\nboosted trees) are the leading mach
predicted_categories = predict_category(new_abstract, loaded_model, loaded_text_vectorizer, invert_multi_hot)
print("Predicted Categories:", predicted_categories)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
Predicted Categories: ['cs.LG' 'cs.AI']
```

In [34]:

```
new_abstract = "Graph neural networks (GNNs) have been widely used to learn vector\nrepresentation of graph-structured data
predicted_categories = predict_category(new_abstract, loaded_model, loaded_text_vectorizer, invert_multi_hot)
print("Predicted Categories:", predicted_categories)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step
Predicted Categories: ['cs.LG']
```



Recommendation System

Data loading and preprocessing

Loading Data: Used pandas to load the Arxiv dataset, with on_bad_lines='skip' to handle any irregularities in the file.

```
arxiv_data = pd.read_csv('/content/arxiv_data_210930-054931.csv', on_bad_lines='skip')
```

Preprocessing: Dropped irrelevant columns (terms, abstracts) to focus only on paper titles, which will be used for recommendation.

```
arxiv_data.drop(columns=['terms', 'abstracts'], axis=1, inplace=True)
```

Sentence Transformer Model for Embeddings

Model Selection: Selected the all-MiniLM-L6-v2 Sentence Transformer for generating embeddings.

- It's a compact model that converts text into 384-dimensional embeddings, capturing the semantic meaning of the text.

Embedding Creation: Encoded all paper titles into embeddings (one embedding per title).

Output Verification: Checked the shape of embeddings to confirm that each title was converted into a 384-dimensional vector.

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
sentences = arxiv_data['titles']
embeddings = model.encode(sentences)
```

Saving Data: Used `pickle` to save embeddings, sentences, and the model. This helps avoid recomputing embeddings each time, saving processing time.

Cosine Similarity: Used cosine similarity (util.cos_sim) to measure how close the embeddings of different titles are to the input title.

```
from sentence_transformers import util
util.cos_sim(embeddings, myemb)
```

Building the Recommendation Function

Function Purpose: Takes a title input and returns the top 10 similar paper titles.
Encoding Input: Encodes the input title using the preloaded model.
Cosine Similarity Calculation: Computes cosine similarity between the input title and all stored embeddings.
Top-K Selection: Uses torch. topk to get the 20 most similar papers and filters to the top 10 unique recommendations.
Return Value: Returns a list of recommended titles, filtered for uniqueness to avoid duplicates.

```
def recommendation(input_paper, threshold=0.9):
    cosine_scores = util.cos_sim(embeddings, recc_model.encode(input_paper))

    top_similar_papers = torch.topk(cosine_scores, dim=0, k=20, sorted=True)

    papers_list = []
    seen_titles = set()

    for i in top_similar_papers.indices:
        title = sentences[i.item()]
        if title not in seen_titles:
            papers_list.append(title)
            seen_titles.add(title)

        if len(papers_list) == 10:
            break

    return papers_list
```

Demo

In [17]:
```
# usage = (Review of deep learning: concepts, CNN architectures, challenges, applications, future directions)
input_paper = input("enter paper title - ")
recommended_papers = recommendation(input_paper)

recommended_papers
```

```
enter paper title - Review of deep learning: concepts, CNN architectures, challenges, applications, future directions
```

Out[17]:
```
['A Review of Deep Learning with Special Emphasis on Architectures, Applications and Recent Trends',
 'Review of Deep Learning',
 'Deep Convolutional Neural Networks: A survey of the foundations, selected improvements, and some current applications',
 'A Survey of the Recent Architectures of Deep Convolutional Neural Networks',
 'A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects',
 'Deep Learning for Medical Image Processing: Overview, Challenges and Future',
 'A Survey on Understanding, Visualizations, and Explanation of Deep Neural Networks',
 'Training Deep Architectures Without End-to-End Backpropagation: A Brief Survey',
 'A Non-Technical Survey on Deep Convolutional Neural Network Architectures',
 'Object Detection with Deep Learning: A Review']
```

Running the code

Packages required -

- pandas
- numpy
- torch
- sentence-transformers
- scikit-learn
- matplotlib
- tensorflow

Dependencies required

- Python
- Jupyter notebook

- pip install pandas numpy torch sentence-transformers scikit-learn tensorflow matplotlib

github link - **https://github.com/33SN0W/nlp_project**

Presentation link -
https://docs.google.com/presentation/d/1vWOKkGaxIlgAsafd-ut22htVnPdweRe-x73QGARpFCM/edit?usp=sharing